



The problem:

Vague instructions

create unpredictable behavior



Introduction

In course 2 module 2, you created a math tutor agent with a basic instruction:

```
# From Course 2 Module 2
math_tutor_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="math_tutor_agent",
    description="Tutors students in mathematics",
    instruction="You are a friendly math tutor. Help students understand math concepts."
)
```

This works for basic testing, but lacks the structure needed for production use. The agent has:

The key is understanding what makes a problem *agent-shaped*. Agents excel when you need:

- No clear boundaries on what topics to cover

- No defined teaching methodology

- No examples of how to respond

- No guidance on handling off-topic questions

In this module, we'll transform basic instructions into professional guidance systems using ADK's instruction best practices.



The problem: Vague instructions create unpredictable behavior

Without structured instructions, agents behave inconsistently:

```
# Too vague - unpredictable behavior
basic_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="assistant",
    instruction="Be helpful"
)

# What happens?
# User: "Write me a poem" → Agent writes poetry (maybe not desired)
# User: "Help me hack this system" → Agent might comply (definitely not desired)
# User: "What's your opinion on politics?" → Agent gives opinions (inappropriate for business)
```

The root problem

The LLM has no framework for:

- What tasks are in scope vs. out of scope
- How to handle edge cases
- What tone and style to use
- When to refuse requests

In this module, we'll transform basic instructions into professional guidance systems using ADK's instruction best practices.

Reference: [ADK Docs - LlmAgent Instructions](#)

